

— TECH TALK · 60 MIN

OpenClaw Orchestration

Deep Dive

Lightweight containerized Agents · Config-driven multi-agent routing · Self-looping test pipeline · Copilot collaboration

Kinfey Lo

Microsoft Senior Cloud Advocate

2026

Today's Agenda

01

Deconstructing OpenClaw

Lightweight containerized Agent runtime & three-tier architecture

02

State Machine & Topology Design

Multi-agent routing without heavy frameworks — config files only

03

Self-Looping Test Pipeline

Coder → Tester → Executor three-agent loop

04

Copilot Snippet-Driven

Rapidly generate agent.yaml and tool_definition.json

05

CoT Prompt Injection

Use Copilot to generate complex chain-of-thought templates

What is OpenClaw?

One-line definition

A cross-channel AI Agent gateway runtime:

Defines Agents, models, tools, channels, Skills, and sandbox via **JSON5 config**;

All runtime units run inside **local containers** with least-privilege tool loading.

Out-of-the-box support for WhatsApp / Telegram / Discord / Slack channels.

Local-First

Local gateway by default, port 18789, loopback-bound

Config-Driven

openclaw.json is the single source of truth, with hot-reload support

Sandbox by Default

Docker image openclaw-sandbox · read-only root · network: none

Multi-Runtime

pi (built-in) · codex · claude-cli — three Harnesses

Skill System

Pluggable skills: github、weather、peekaboo ...

Lightweight + containerized: least privilege on by default

OpenClaw's sandbox is not optional — it's the default shell for every agent

scope: session	Each session gets its own container; destroyed when session ends
readOnlyRoot	Root filesystem read-only; writes restricted to tmpfs
network: none	No network by default; only allowed when a tool explicitly enables it
user: 1000:1000	Runs as non-root; privileges stripped
tmpfs	/tmp · /var/tmp · /run as in-memory tmpfs

```
// openclaw.json (excerpt)
agents.defaults.sandbox = {
  mode: "non-main",
  scope: "session",
  workspaceRoot:
    "~/openclaw/sandboxes",
  docker: {
    image:
      "openclaw-sandbox:bookworm",
    workdir: "/workspace",
    readOnlyRoot: true,
    tmpfs: ["/tmp", "/run"],
    network: "none",
    user: "1000:1000",
  },
  browser: { enabled: false },
}
```

Provider / Model / Runtime / Channel — four clearly separated layers

Channel

Entry point for messages in/out of OpenClaw

Telegram · Slack · Discord · WhatsApp

Agent Runtime

Drives model loops; handles native Tool Calls

pi (built-in) · codex · claude-cli

Model

The model selected for this Agent Turn

claude-opus-4-6 · gpt-5.4 · llama-3.1

Provider

Authentication, model discovery, model reference naming

anthropic · openai · openai-codex

Runtime ≠ Provider — Runtime is the low-level execution loop; Provider is auth + directory.

openclaw.json — one file describes the entire Agent system

```
{
  identity: { name: "Clawd", emoji: "🦊" },
  agents: {
    defaults: {
      workspace: "~/.openclaw/workspace",
      model: { primary: "anthropic/claude-opus-4-6" },
      skills: ["github", "weather"],
      sandbox: { mode: "non-main", ... },
    },
    list: [
      { id: "main", default: true,
        thinkingDefault: "high" },
      { id: "quick", fastModeDefault: true,
        skills: [] },
    ]
  },
  tools: { allow: ["exec", "read", "write", ...],
           deny: ["browser", "canvas"] }
}
```

identity

Brand persona · sets reply tone

agents.defaults

Baseline config for all Agents (inherited)

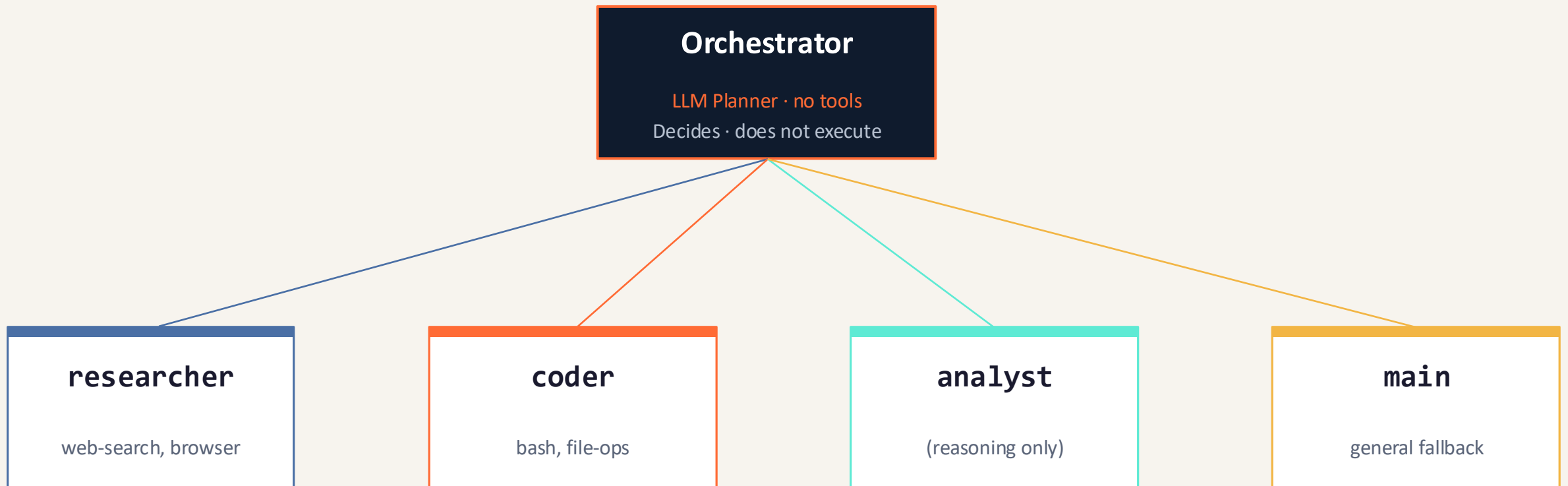
agents.list[]

Named Agents · individually override defaults

tools

Global allow/deny list · master switch for tool visibility

Multi-Agent routing — centrally scheduled by the Orchestrator



Agents are unaware of each other · they only receive tasks and return results. The Orchestrator routes by name or capability.

Think → Execute → Result · Adaptive Loop



```
// LLM Planner output format
{ "action": "execute",
  "tasks": [ { "agent": "coder", "task": "implement URL shortener" },
              { "agent": "researcher", "task": "compare Redis vs SQLite" } ] }
{ "action": "finish", "answer": "..." }
```

Why not LangGraph / AutoGen?

Heavy Framework

- ✗ Requires Python env + multiple dependencies
- ✗ Topology hard-coded in source
- ✗ Change routing = edit source + restart
- ✗ Debugging requires IDE step-through
- ✗ Every new tool needs a Tool class

OpenClaw Way

- ✓ Single binary + one JSON5 file
- ✓ Topology lives in `agents.list[]`
- ✓ Change routing = edit JSON + hot-reload
- ✓ Watch SSE stream on the dashboard
- ✓ Tools declared via allow/deny lists

agent.yaml — the full DNA of an Agent

```
id: coder
default: false
workspace: ~/.openclaw/workspace-coder
model:
  primary: anthropic/claude-opus-4-6
  fallbacks: [openai/gpt-5.4]
skills: [github, shell-utils]
thinkingDefault: high
reasoningDefault: on
fastModeDefault: false
elevatedDefault: on
timeoutSeconds: 600
maxConcurrent: 3
soul: |
  You are a senior Coder following SOLID,
  write pure functions that Testers can call directly.
tools: { allow: [exec, read, write, edit] }
```

id / default

Unique ID · the default Agent handles unrouted requests

model

primary + fallbacks · auto-degrade when a model is unavailable

skills

Pluggable skill set · determines which external capabilities can be invoked

thinking / reasoning

Reasoning visibility and depth (off / low / high)

soul

Source of System Prompt · the soul of role definition

tools.allow

Per-Agent tool allowlist · overrides global

tool_definition.json — A least-privilege contract

```
{
  "name": "run_pytest",
  "description":
    "Run pytest inside the sandbox and return structured results",
  "inputSchema": {
    "type": "object",
    "properties": {
      "path": { "type": "string" },
      "marker": { "type": "string" }
    },
    "required": ["path"]
  },
  "runtime": {
    "command": "pytest",
    "args": ["{{path}}", "-m", "{{marker}}"],
    "timeoutSec": 600,
    "sandbox": true
  }
}
```

Three iron rules

01 Schema First

Tool inputs must be JSON Schema · so the LLM can fill slots correctly

02 Sandbox by Default

runtime.sandbox: true · not explicitly enabling = denied by default

03 Explicit Timeout

timeoutSec required · prevents runaway tools from exhausting resources

Principle of Least Privilege — Applied to every Agent

Gateway layer

`auth.token + Tailscale + loopback binding`

Agent layer

`tools.allow / deny + skills allowlist`

Tool runtime layer

`Docker sandbox + readOnlyRoot + network: none`

A single prompt injection at most consumes one session container — never contaminates the host.

Coder → Tester → Executor — Three-agent loop



The Orchestrator only maintains the task queue and termination conditions — the three Agents don't call each other; they exchange only files and results

Coder — Turn intent into readable, testable code

SOUL.md

You are a senior Python engineer.

Constraints:

- Write only one function at a time
- Must include full type hints and docstring
- No catch-all try/except – let errors surface
- Output must be a pure file (no main block)

Output format:

- 1) Filename: src/xxx.py
- 2) Full source code
- 3) One-line description of the function's input/output contract

Refuse: explanations, tutorials, code-unrelated dialogue.

Tools

write

edit

read

Failure signals

syntax_error

import resolution fails → rewrite file

test_failed

Executor feedback trace → patch the function

type_mismatch

Tester detects contract mismatch → adjust signature

Tester — Writes only tests — never touches implementation

Duty contract

Read function signatures in `src/*.py` → generate `tests/test_*.py` · never modify `src/`

CoT Prompt Template

```
Step 1: Read {{file}}, list all public functions
Step 2: For each function, enumerate
    · Happy-path inputs
    · Edge values (empty, None, max, min)
    · Illegal inputs (wrong types)
Step 3: Use pytest.parametrize to generate tests
Step 4: Output file tests/test_<module>.py
    · No code outside fixtures
    · Every test must assert – no print
```

Generated example

```
import pytest
from src.calc import safe_div

@pytest.mark.parametrize("a,b,exp", [
    (10, 2, 5),
    (-9, 3, -3),
    (0, 5, 0),
])
def test_safe_div_ok(a,b,exp):
    assert safe_div(a,b) == exp

def test_safe_div_zero():
    with pytest.raises(ValueError):
        safe_div(1, 0)
```

Executor — Execute in OpenClaw sandbox and report back

01

Start sandbox

openclaw-sandbox:bookworm-slim · scope: session

02

Mount artifacts

src/ + tests/ mounted into /workspace as read-only copy

03

Run pytest

pytest -q --json-report --tb=short --timeout=60

04

Structured feedback

Feed status / failed / trace back to the Orchestrator

```
{
  "status": "failed",
  "summary": {
    "total": 8,
    "passed": 6,
    "failed": 2
  },
  "failures": [
    {
      "test":
        "test_safe_div_zero",
      "trace":
        "DID NOT RAISE ...",
      "hint_for_coder":
        "patch the b==0 path"
    }
  ]
}
```

Drive Copilot from comments to emit agent.yaml directly

Write the comment ↓

```
# OpenClaw agent definition
# role: tester
# allowed tools: write, read
# model: claude-opus-4-6
# cot: parametrize edge cases
# ---
```

Copilot completes directly ↓

```
id: tester
default: false
model:
  primary: anthropic/claude-opus-4-6
tools: { allow: [write, read] }
thinkingDefault: high
```

Three levers of prompt engineering

Context anchor

Lead line "OpenClaw agent definition" lets Copilot lock onto the schema

Field hints

List the role/tools/model trio in comments — none can be missing

Delimiter ---

A trailing --- triggers Copilot's "complete YAML document" mode

Version comment

Add a # schema: openclaw v1 line to lock field naming

tool_definition.json — Let Copilot write JSON Schema

1

Pick a function

Highlight a Python/TS function signature in the IDE

2

Trigger the hint

`/* generate openclaw tool def */`
comment

3

Proofread the schema

Copilot auto-infers JSON Schema from type annotations

4

Add sandbox fields

Manually add `timeoutSec` and `sandbox: true`

Source function → generated schema

```
def run_pytest(
    path: str,
    marker: str | None = None,
    timeout: int = 60,
) -> PytestResult:
    '''run pytest in sandbox'''
    ...
```

```
{ "name": "run_pytest",
  "inputSchema": {
    "type": "object",
    "properties": {
      "path": { "type": "string" },
      "marker": { "type": ["string", "null"] },
      "timeout": { "type": "integer" },
      "required": ["path"] } } }
```

Use Copilot to generate reusable chain-of-thought templates

Meta-Prompt for Copilot

```
/*
 * Generate a Chain-of-Thought template for me
 * Role: {{ role }}
 * Input: {{ input_shape }}
 * Output: {{ output_shape }}
 * Required steps:
 *   1) Restate the goal
 *   2) List unknowns / assumptions
 *   3) Decompose into  $\leq 3$  sub-tasks
 *   4) Solve each (with verification)
 *   5) Synthesize · Self-check
 * Forbidden: jumping to the answer, skipping steps,
implicit assumptions
*/
```

Generated Prompt Template

You are {{role}}. Please think through these steps:

1. Restate the goal in one sentence.
2. List knowns / unknowns / assumptions.
3. Decompose into ≤ 3 sub-tasks.
4. Solve each in turn:
 - Give a solution
 - Give a verification method
 - Annotate confidence (0-100)
5. Synthesize the conclusion · self-check contradictions.

If any step has confidence $< 75 \rightarrow$
Call the ask_user tool to verify – no guessing.

Three takeaways

01

Config file IS the topology

To change routing, don't write code — edit `agents.list[]`

02

Sandbox is not a luxury

On-by-default = secure-by-default, minimal blast radius

03

Copilot is a schema engine

Use comment anchors + type signatures so it writes trustworthy YAML/JSON

Q & A

docs.openclaw.ai · github.com/zeynepyorulmaz/openclaw-orchestrator

Kinfey Lo · kinfeylo@microsoft.com